

Graph Library: Graph Containers

Document #: **P3131r0**
Date: 2024-02-05
Project: Programming Language C++
Audience: Library Evolution
SG19 Machine Learning
SG14 Game, Embedded, Low Latency
SG6 Numerics
Revises: P1709r5

Reply-to: Phil Ratzloff (SAS Institute)
phil.ratzloff@sas.com
Andrew Lumsdaine
lumsdaine@gmail.com

Contributors: Kevin Deweese
Muhammad Osama (AMD, Inc)
Jesun Firoz
Michael Wong (Codeplay)
Jens Maurer
Richard Dosselmann (University of Regina)
Matthew Galati (Amazon)

1 Getting Started

This paper is one of several interrelated papers for a proposed Graph Library for the Standard C++ Library. The Table 1 describes all the related papers.

Paper	Status	Description
P1709	Inactive	Original proposal, now separated into the following papers.
P3126	Active	Overview , describing the big picture of what we are proposing.
P3127	Active	Background and Terminology providing the motivation, theoretical background and terminology used across the other documents.
P3128	Active	Algorithms covering the initial algorithms as well as the ones we'd like to see in the future.
P3129	Active	Views has helpful views for traversing a graph.
P3130	Active	Graph Container Interface is the core interface used for uniformly accessing graph data structures by views and algorithms. It is also designed to easily adapt to existing graph data structures.
P3131	Active	Graph Containers describing a proposed high-performance <code>compressed_graph</code> container. It also discusses how to use containers in the standard library to define a graph, and how to adapt existing graph data structures.

Table 1: Graph Library Papers

Reading them in order will give the best overall picture. If you're limited on time, you can use the following guide to focus on the papers that are most relevant to your needs.

Reading Guide

- If you're **new to the Graph Library**, we recommend starting with the *Overview* paper (P3126) to understand focus and scope of our proposals.
- If you want to **understand the theoretical background** that underpins what we're doing, you should read the *Background and Terminology* paper (P3127).
- If you want to **use the algorithms**, you should read the *Algorithms* paper (P3128) and *Graph Containers* paper (P3131).
- If you want to **write new algorithms**, you should read the *Views* paper (P3129), *Graph Container Interface* paper (P3130) and *Graph Containers* paper (P3131). You'll also want to review existing implementations in the reference library for examples of how to write the algorithms.
- If you want to **use your own graph container**, you should read the *Graph Container Interface* paper (P3130) and *Graph Containers* paper (P3131).

2 Revision History

P3131r0

- Split from P1709r5. Added *Getting Started* section.
- Move text for graph data structures created from std containers from Graph Container Interface to Container Implementation paper.
- GCI overloads are no longer required for adjacency lists constructed with standard containers. Data structures that follow the pattern `random_access_range<forward_range<integral>>` and `random_access_range<forward_range<tuple<integral, ...>>>` are automatically recognized as an adjacency list, including containers from non-standard libraries. The `integral` value is used as the `target_id`.

3 Naming Conventions

Table 2 shows the naming conventions used throughout the Graph Library documents.

Template Parameter	Type Alias	Variable Names	Description
G			Graph
	<code>graph_reference_t<G></code>	<code>g</code>	Graph reference
GV		<code>val</code>	Graph Value, value or reference
V	<code>vertex_t<G></code>		Vertex
	<code>vertex_reference_t<G></code>	<code>u,v,x,y</code>	Vertex reference. <code>u</code> is the source (or only) vertex. <code>v</code> is the target vertex.
VId	<code>vertex_id_t<G></code>	<code>uid,vid,seed</code>	Vertex id. <code>uid</code> is the source (or only) vertex id. <code>vid</code> is the target vertex id.
VV	<code>vertex_value_t<G></code>	<code>val</code>	Vertex Value, value or reference. This can be either the user-defined value on a vertex, or a value returned by a function object (e.g. <code>VVF</code>) that is related to the vertex.
VR	<code>vertex_range_t<G></code>	<code>ur,vr</code>	Vertex Range
VI	<code>vertex_iterator_t<G></code>	<code>ui,vi</code>	Vertex Iterator. <code>ui</code> is the source (or only) vertex.
		<code>first,last</code>	<code>vi</code> is the target vertex.
VVF		<code>vvf</code>	Vertex Value Function: <code>vvf(u) → vertex value</code> , or <code>vvf(uid) → vertex value</code> , depending on requirements of the consume algorithm or view.
VProj		<code>vproj</code>	Vertex descriptor projection function: <code>vproj(x) → vertex_descriptor<VId,VV></code> .
	<code>partition_id_t<G></code>	<code>pid</code>	Partition id.
		<code>P</code>	Number of partitions.
PVR	<code>partition_vertex_range_t<G></code>	<code>pur,pvr</code>	Partition vertex range.
E	<code>edge_t<G></code>		Edge
	<code>edge_reference_t<G></code>	<code>uv,vw</code>	Edge reference. <code>uv</code> is an edge from vertices <code>u</code> to <code>v</code> . <code>vw</code> is an edge from vertices <code>v</code> to <code>w</code> .
EId	<code>edge_id_t<G></code>	<code>eid,uvid</code>	Edge id, a pair of vertex_ids.
EV	<code>edge_value_t<G></code>	<code>val</code>	Edge Value, value or reference. This can be either the user-defined value on an edge, or a value returned by a function object (e.g. <code>EVF</code>) that is related to the edge.
ER	<code>vertex_edge_range_t<G></code>		Edge Range for edges of a vertex
EI	<code>vertex_edge_iterator_t<G></code>	<code>uvi,vwi</code>	Edge Iterator for an edge of a vertex. <code>uvi</code> is an iterator for an edge from vertices <code>u</code> to <code>v</code> . <code>vwi</code> is an iterator for an edge from vertices <code>v</code> to <code>w</code> .
EVF		<code>evf</code>	Edge Value Function: <code>evf(uv) → edge value</code> , or <code>evf(eid) → edge value</code> , depending on the requirements of the consuming algorithm or view.
EProj		<code>eproj</code>	Edge descriptor projection function: <code>eproj(x) → edge_descriptor<VId,Sourced,EV></code> .
PER	<code>partition_edge_range_t<G></code>		Partition Edge Range for edges of a partition vertex.

Table 2: Naming Conventions for Types and Variables

4 compressed_graph

`compressed_graph` is a graph container being proposed for the standard library. It is a high-performance data structure that uses [Compressed Sparse Row](#) format to store its vertices, edges and associated values. Once constructed, vertices and edges cannot be added or deleted but values on vertices and edges can be modified.

The following listing shows the prototype for the `compressed_graph`. Only the members shown for `compressed_graph` are public. No other member functions or types are exposed as part of the standard. All other types are only accessible through the types and functions in the Graph Container Interface. Multiple partitions (multi-partite) can be defined by passing the number of partitions in a constructor.

Implements <code>load_graph</code> ? Yes	Append vertices? No	vertex_id assignment: Contiguous
Implements <code>load_vertices</code> ? Yes	Append edges? No	Vertices range: Contiguous
Implements <code>load_edges</code> ? Yes		Edge range: Contiguous
Implements <code>load_partition</code> ? Yes		

```
template <class EV = void, // Edge Value type
         class VV = void, // Vertex Value type
         class GV = void, // Graph Value type
         integral VId = uint32_t, // vertex id type
         integral EIndex = uint32_t, // edge index type
         class Alloc = allocator<VId>> // for internal containers
class compressed_graph {
public:
    compressed_graph();
    explicit compressed_graph(size_t num_partitions); // multi-partite
    compressed_graph(const compressed_graph&);
    compressed_graph(compressed_graph&&);
    {tilde}compressed_graph();

    compressed_graph& operator=(const compressed_graph&);
    compressed_graph& operator=(compressed_graph&&);
}
```

1 Mandates:

- (1.1) — Vertices and edges cannot be appended to an existing partition in an existing graph, but they can be added to a new partition.

2 Preconditions:

- (2.1) — The `VId` template argument must be able to store a value of $|V|+1$, where $|V|$ is the number of vertices in the graph. The size of this type impacts the size of the *edges*.
- (2.2) — The `EIndex` template argument must be able to store a value of $|E|+1$, where $|E|$ is the number of edges in the graph. The size of this type impact the size of the *vertices*.

3 Effects:

- (3.1) — When `EV`, `VV`, or `GV` are `void`, no extra memory overhead is incurred for it.
- (3.2) — The `VId` and `EIndex` template arguments impact the internal storage requirements and performance. The default of `uint32_t` is sufficient for most graphs and provides a good balance between storage and performance.

4 Remarks:

- (4.1) — The default allocator type of `allocator<VId>` is rebound for different internal containers.

5 Using Existing Graph Data Structures

Reasonable defaults have been defined for the GCI functions to minimize the amount of work needed to adapt an existing graph data structure to be used by the views and algorithms.

There are two cases supported. The first is for the use of standard containers to define the graph and the other is for a broader set of more complicated implementations.

5.1 Using Standard Containers for the Graph Data Structure

For example this we'll use `G = vector<forward_list<tuple<int,double>>>` to define the graph, where `g` is an instance of `G`. `tuple<int,double>` defines the target `_id` and weight property respectively. We can write loops to go through the vertices, and edges within each vertex, as follows.

```
using G = vector<forward_list<tuple<int,double>>>;
auto weight = [&g](edge_t& uv) { return get<1>(uv); }

G g;
load_graph(g, ...); // load some data

// Using GCI functions
for(auto&& [uid, u] : vertices(g)) {
    for(auto&& [vid, uv]: edges(g,u)) {
        auto w = weight(uv);
        // do something...
    }
}
```

Note that no function override was required and is a special case when the outer range is a `random_access_range` and inner inner range is a `forward_range`, and the value type of the inner range is either `integral` or `tuple<integral, ...>`. This extends to any range type. For instance, `boost::containers` can be used just as easily as `std` containers.

Function or Value	Concrete Type
<code>vertices(g)</code>	<code>vector<forward_list<tuple<int,double>>></code> (when <code>random_access_range<G></code>)
<code>u</code>	<code>forward_list<tuple<int,double>></code>
<code>edges(g,u)</code>	<code>forward_list<tuple<int,double>></code> (when <code>random_access_range<vertex_range_t<G>></code>)
<code>uv</code>	<code>tuple<int,double></code>
<code>edge_value(g,uv)</code>	<code>tuple<int,double></code> (when <code>random_access_range<vertex_range_t<G>></code>)
<code>target_id(g,uv)</code>	<code>integral</code> , when <code>uv</code> is either <code>integral</code> or <code>tuple<integral,...></code>

Table 3: Types When Using Standard Containers

5.2 Using Other Graph Data Structures

For other graph data structures more function overrides are required. Table 4 shows the common function overrides anticipated for most cases, keeping in mind that all functions can be overridden. When they are defined they must be in the same namespace as the data structures.

Acknowledgements

Phil Ratzloff's time was made possible by SAS Institute.

Portions of *Andrew Lumsdaine's* time was supported by NSF Award OAC-1716828 and by the Segmented Global Address Space (SGAS) LDRD under the Data Model Convergence (DMC) initiative at the U.S. Department of Energy's Pacific Northwest National Laboratory (PNNL). PNNL is operated by Battelle Memorial Institute under Contract DE-AC06-76RL01830.

Function	Comment
<code>vertices(g)</code>	
<code>edges(g,u)</code>	
<code>target_id(g,uv)</code>	
<code>edge_value(g,uv)</code>	If edges have value(s) in the graph
<code>vertex_value(g,u)</code>	If vertices have value(s) in the graph
<code>graph_value(g)</code>	If the graph has value(s)
----- When edges have the optional source_id on an edge -----	
<code>source_id(g,uv)</code>	
----- When the graph supports multiple partitions -----	
<code>partition_count(g)</code>	
<code>partition_id(g,u)</code>	
<code>vertices(g,u,pid)</code>	

Table 4: Common CPO Function Overrides

Michael Wong's work is made possible by Codeplay Software Ltd., ISOCPP Foundation, Khronos and the Standards Council of Canada.

Muhammad Osama's time was made possible by Advanced Micro Devices, Inc.

The authors thank the members of SG19 and SG14 study groups for their invaluable input.